Module-I
Lecture-I
Introduction to Digital VLSI Design Flow

1. Introduction VLSI Design

The functionality of electronics equipments and gadgets has achieved a phenomenal growth over the last two decades while their physical sizes and weights have come down drastically. The major reason is due to the rapid advances in integration technologies, which enables fabrication of millions of transistors in a single Integrated Circuit (IC) or chip. IC (used interchangeably with "chip" in this lecture) is a device having multiple transistors with interconnects manufactured on a single silicon substrate. Integration with a complexity of 10's of transistors is called Small Scale Integration, with 100's is Medium Scale Integration (MSI), with 1000's is Large Scale Integration (LSI), with 10,000 it is Very Large Scale Integration (VLSI) [1]. As a very huge number of components can be integrated in a single IC fabricated using VLSI technology, the variant of functionalities provided by such ICs can be as large as those which were provided by thousands of LSI ICs. In other words, systems of systems can be implemented in a VLSI IC. However, with this rise in functionality of VLSI ICs, design problem has become huge and complex. To address this complexly issue, after the design specifications are complete almost all the other steps are automated using CAD tools. However, even designs automated using CAD tools may have bugs. Also, due to extremely large size of the design space it is not possible to verify correctness of the design under all possible situations. So technique are required that can verify, without exercising exhaustive input-output combinations, that the design meets all the input specifications; this technique is called formal verification. Finally, when the design meets all specifications (as it is formally verified) it is manufactured and sent to market. In VLSI designs as millions of transistors are packed into a single chip, the device and interconnect sizes are extremely small and so are the inert-component distances. This leads to manufacturing defects and all the chips need to be physically tested by giving input signals from a pattern generator and comparing responses using a logic analyzer; this process is called Testing. So, in the process of manufacturing a VLSI IC there are

three broad steps: DESIGN-VERIFICATION-TEST. There are algorithms and CAD tools which automate these three steps.

VLSI ICs can be divided into analog, digital or mixed-signal (both analog and digital on the same chip) based on their functionality. Digital ICs can contain logic gates, flip-flops, multiplexers, and other circuits which work using binary mathematics to process "one" and "zero" signals. Analog ICs, such as current mirrors, voltage followers, filters, OPAMPs etc. work by processing continuous signals. They perform functions like amplification, active filtering, demodulation etc. When single IC has both analog and digital components it is called mixed signal IC e.g, Analog to Digital Converter (ADC). The automation algorithms and CAD tools are mainly available for digital ICs because digital circuits comprise millions of components and transformation of design specifications to silicon implementation can be accomplished using logical procedures (which can be converted to algorithms and tools) [2]. However, most of the analog circuits comprise less than hundred devices and its design is like an “art” which is best performed by designers with “aid” of some CAD tools (which provides feedback to designer if the manual design is progressing fine etc.) [2]. In this course we will deal only with digital VLSI circuits. Henceforth, in this course VLSI IC would imply digital VLSI ICs only and whenever we want to discuss about analog or mixed signal ICs it will be mentioned explicitly. Also, in this course the terms ICs and chips would mean VLSI ICs and chips.

This course is concerned with algorithms required to automate the three steps “DESIGN-VERIFICATION-TEST” for Digital VLSI ICs. Although there are individual courses catering to each step, in this course we will try to give a complete picture of algorithms required to automate the entire flow “DESIGN-VERIFICATION-TEST”. This course will give you a comprehensive idea of algorithms required in CAD tools for the whole digital VLSI flow, however, to cover the entire flow at certain points we will go to a limited depth and provide references to the details.

In the first lecture we will introduce the entire flow of “DESIGN-VERIFICATION-TEST” using a simple example, in brief. Also, we will not go into the algorithms required to automate the steps, rather we will point the requirements of the algorithms.

2. Flow of VLSI "DESIGN-VERIFICATION-TEST" using an example

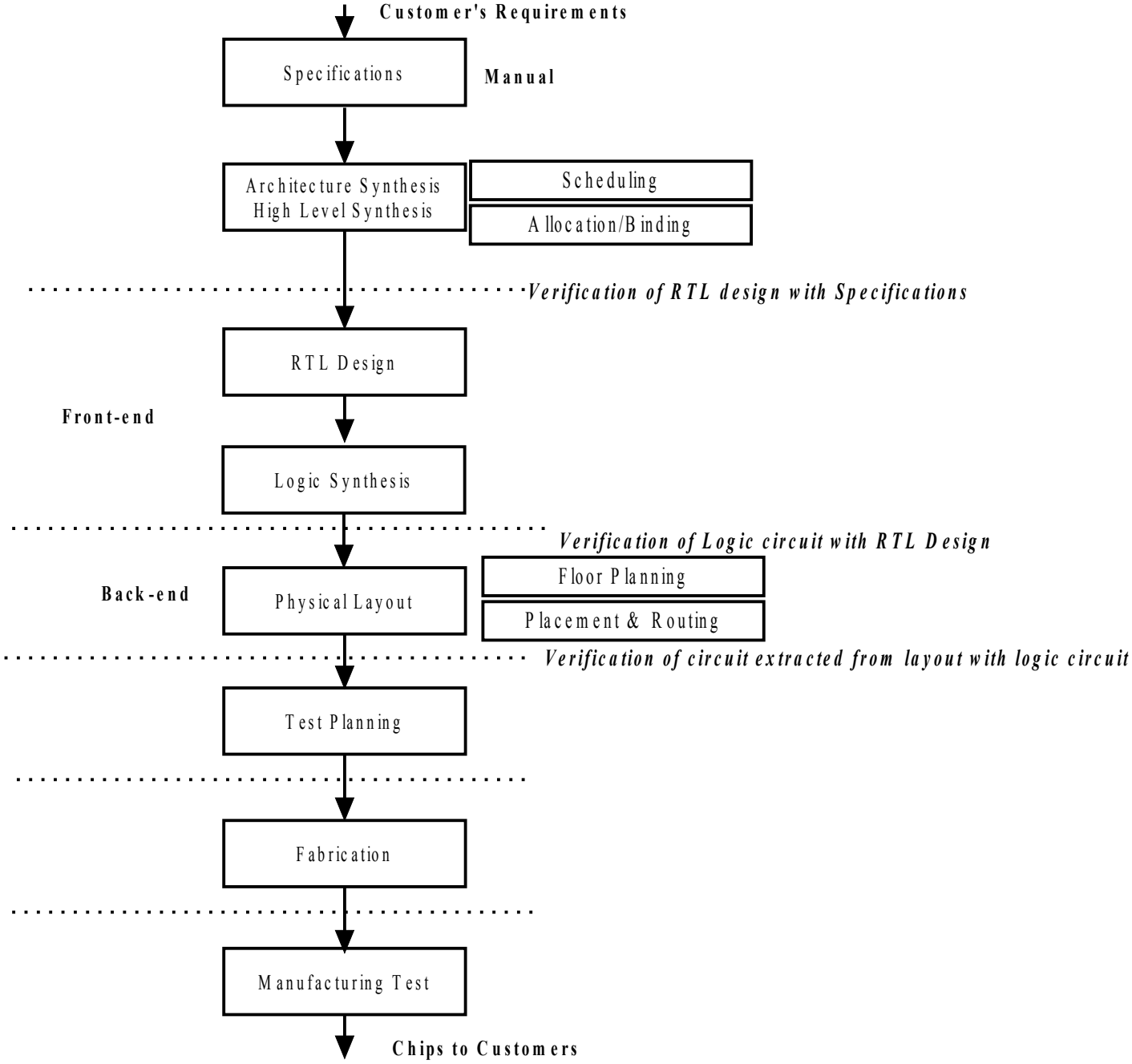


Figure 1. VLSI Design, Verification and Test Flow

Figure 1 illustrates a typical VLSI DESIGN-VERIFICATION-TEST flow.

Step1: Specification Design

In a typical VLSI flow, we start with system specifications, which is nothing but technical representation of design intent. To explain the flow, the following example will be used through this section.

Example:

Specification: out1=a+b; out2=c+d; where a,b,c,d are single bit inputs and out1,out2 are two bit outputs (sum and carry).

Step 2: High level Synthesis

High-level synthesis (HLS) algorithms are used to convert specifications into Register Transfer Level (RTL) circuits.

HLS, sometimes referred to as architectural synthesis is an automated design procedure that interprets an algorithmic description of the design intent and creates hardware at RTL that implements that behavior [3]. The input to a HLS tool is design intent written in some high level hardware definition language like SystemC, System Verilog etc. The HLS tool first schedules the computations (required to meet the specifications) at different control steps. The computations scheduled at each control step contains operations which can be performed in a single clock cycle in the hardware. Following that, depending on availability of hardware units and time constraints, the scheduled computations (comprising instructions and variables) are allocated and binded to the hardware units like adders, multipliers, multiplexors, registers, wires etc.

Example:

In the example there are two operations (addition of single bit numbers) and none of them depend on each other. So both the operations can be scheduled in a single control step. However, if there are dependencies e.g., out1=a+b; out2=out1+d; then "out1=a+b;" is scheduled in 1st control step whereas "out2=out1+d;" is scheduled in 2nd control step. Figure 2 illustrates scheduling of the operations of the example.

Now depending on availability of hardware resources and time constraints the scheduled operators and variables are allocated and binded to hardware units. Let

there be one adder and two registers in the library. So the two operations (addition) of the example, even if scheduled in one control step, cannot be allocated to the single adder. Similarly, the four variables cannot be allocated to two registers. In the running example with the given resource constraints, the two operations can be done in two control steps: Step 1- variable *a* is allocated to Register1, variable *b* is allocated to Register2 and operation "out1=Register1+Register2;" is allocated to adder; Step 2- variable *c* is allocated to Register1, variable *d* is allocated to Register2 and operation "out2=Register1+Register2;" is allocated to adder. Figure 3 illustrates allocation and binding (of variables and operations) for the running example when there is one adder and two registers in the library.

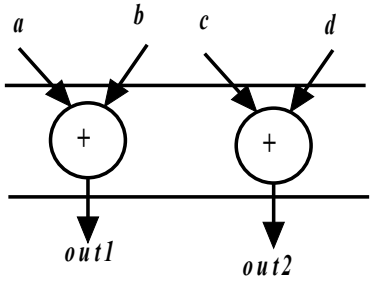


Figure 2. Scheduling of the operations for the example

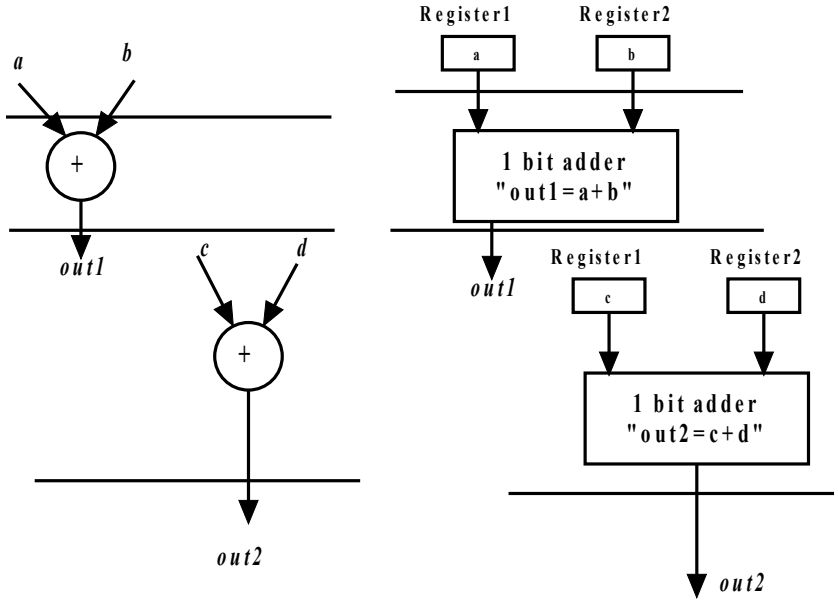


Figure 3. Allocation of operators and variables to hardware units (1 adder + 2 Registers)

However, if there are two adders and four registers in the library then both the operations can be carried out in one control step. Figure 4 illustrates binding of variables and operations for the running example for such a case.

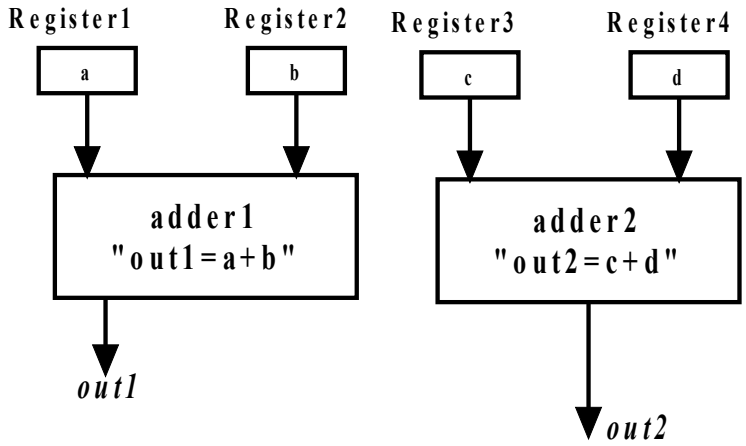


Figure 4. Allocation of operators and variables to hardware units (2 adders + 4 Registers)

Finally, based on allocation and binding, the control unit is to be designed (at high level) in this step. If the allocation/binding is according to Figure 4, the control is trivial. However, if the allocation is according to Figure 3, then the control circuit needs to provide signals that can do multiplexing between *a* and *c, b* and *d;* in 1st control step, *a* should be fed to Register1 and *b* should be fed to Register2, while, in 2nd control step, *c* should be fed to Register1 and *d* should be fed to Register2. Figure 5 illustrates the block diagram where control modules are added after allocation and binding (Figure 3). It may be noted that *control* signal is not available as an external pin which can be controlled by the user. "*Control"* is connected to some signal generated by the system, which alternates in every control step thereby making its value 0 in 1st step and 1 in the 2nd.

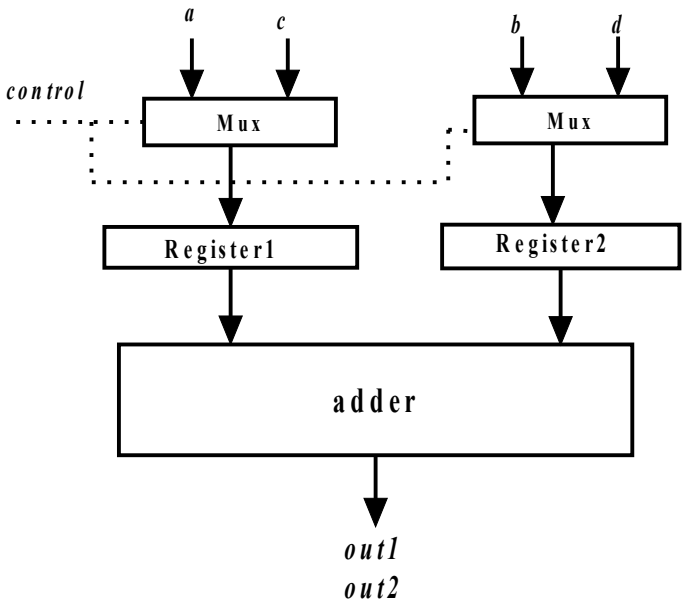


Figure 5. Block diagram with control modules added after allocation and binding

The HLS tool generates output comprising, (i) operations-variables allocated-binded to hardware units and (ii) control modules. The output of HLS tool is called Register Transfer Level (RTL) circuit because data flow, data operations and control flow are captured between registers. After HLS, RTL circuits are transformed into logic gate level implementation; the step is called logic synthesis.

Before the staring of logic synthesis, one needs to verify if the RTL is equivalent to the specifications. In the running example, we can verify by applying all possible input conditions of *a,b,c,d* (along with *control*, if RTL is as per Figure 5) to the RTL and checking if *out1* and *out2* are as expected. However, if the RTL has about hundreds of inputs then exercising all possible inputs is impossible because of the exponential complexity (i.e., if there are *n* inputs then all possible input combinations are 2^n). So we need to have formal verification methods which verify equivalence of RTL with input specifications. Broadly speaking, for formal verification we need to model the RTL circuit and the specifications using some formal modeling techniques and verify that both of them are equivalent. In other words, equivalence is determined without applying inputs. Figure 6 illustrates

Control and Data Flow Diagram (CDFG) [4], a formal modeling, to capture the RTL of Figure 5. Figure 7 illustrates Finite State Machine (FSM) [4] to model the control logic of the RTL of Figure 5. This example being very simple, we can see that both specifications and the model are equivalent. Formal techniques for checking equivalence can be found in [5] and will be elaborated in "VERIFICATION" section of the course.

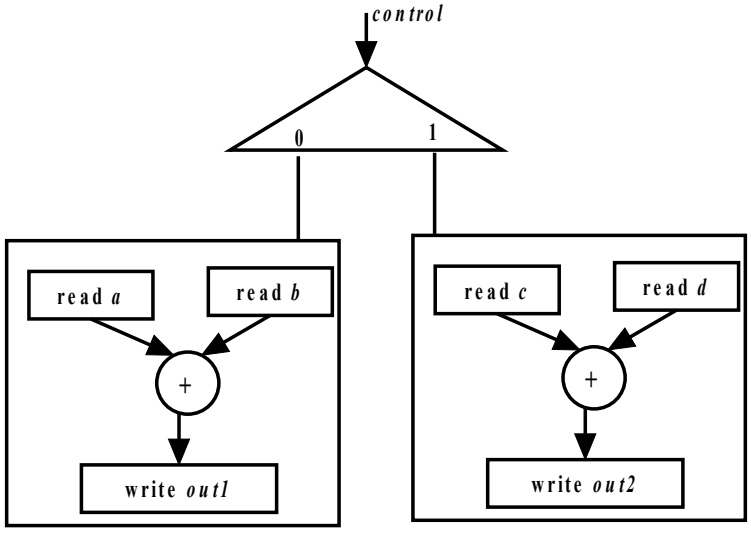


Figure 6. CDFG of the RTL circuit of Figure 5

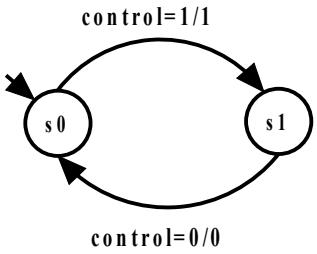


Figure 7. FSM for the control logic of RTL circuit shown in Figure 5

Step 3: Logic Synthesis

After the RTL is verified to be equivalent to system specification, logic synthesis is performed by CAD tools. In logic synthesis all blocks of the RTL circuit is transformed into logic gates and flip-flops. For the running example all the blocks namely, adder, multiplexers, control logic etc. need to be synthesized to logic gates. Here, we will illustrate synthesis only for the adder module and for the rest, similar procedure holds. We will discuss a typical procedure here in brief and details will be explained in the "DESIGN" module of the course.

We first determine the Boolean function of the adder module, in terms of mean terms. Table 1 illustrates the input –output of the adder module.

Table 1. Input –output of the adder module

a	*b*	*Out1(sum)*	*Out1(carry)*
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

From the table we have Boolean equations for

$Out1(sum) = a\bar{b} + \bar{a}b$ and $Out1(carry) = a.b$

After the equations are obtained they need to be minimized so that the circuit can be implemented using minimal number of gates. Karnaugh map, Quine–McCluskey algorithm etc. [6] are some standard techniques to minimize Boolean functions. In this example of the adder, the equations are already minimized and can be directly converted to Boolean gate implementation as shown in Figure 8. Karnaugh map and Quine–McCluskey techniques work well if the number of inputs is less. However, in case of practical VLSI circuits the number of inputs are in orders of hundreds, so minimization is carried out using heuristics techniques, which will be discussed in the "DESIGN" module of the course.

Again equivalence of logic synthesis output should be established with RTL design.

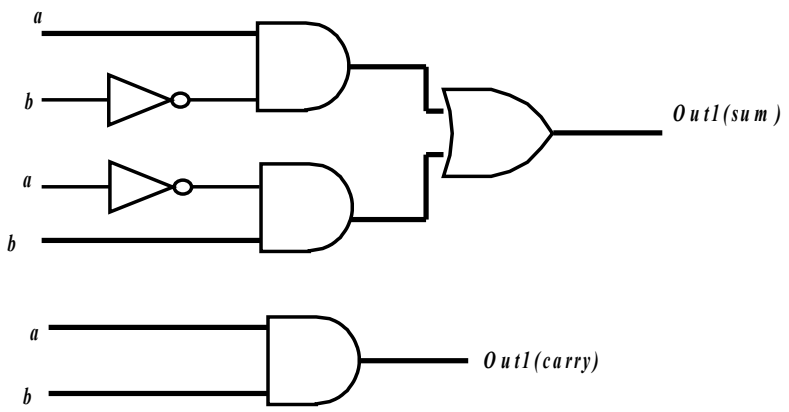


Figure 8. Circuit after logic synthesis for the adder block (of Figure 5)

Step 4: Backend

Once the logic level output of the circuit is obtained we move to backend phase of the design process. Broadly speaking, in backend we start with a software version of the silicon die where the chip will be finally fabricated. In the process, we first make a broad plan regarding placement of gates, flip-flops etc. (output of logic synthesis) in appropriate places in the software representation of the chip; this process is called Floorplan. Following that, we decide on the exact locations in the die (software representation) where the circuit components are placed; this is called Placement. Once placement is done, we make the required interconnections (as given in the logic circuit) among the gates that are placed in exact positions in the die; this process is called routing. The main aim of Floorplan, Placement and Routing is to minimize silicon area, minimize signal delays, well distributed power flow etc., subject to accommodating all the components of the chip and adhere to the design check rules [7].

In this course we will not discuss the Backend phase of design flow; for details the reader is refereed to [7].

Again equivalence of output of Backend process should be established with logic design. In this process we extract the logic circuit from the placed and routed circuit in the die (software representation) and equivalence is verified against logic design (output of logic synthesis).

Step 5: Test Planning

As discussed, in VLSI designs millions of transistors are packed into a single chip, thereby leading to manufacturing defects. So all chips need to be physically tested by providing input signals from a pattern generator and comparing responses using a logic analyzer. As in the case of verification, testing by applying all possible input combinations is prohibitive, due to curse of dimensionality problem. The testing problem is more time hungry than verification because all chips need to be tested while only "one" design is to be verified. Testing by applying all possible input combinations is called exhaustive functional testing, which is avoided because of prohibitive time requirements.

Testing is therefore done based on "structure" of the circuit and is called structural testing. In structural testing we first decide on set of faults that can occur, called Fault Models; stuck-at, bridging etc. are some well known fault models. Then we apply only those inputs which are required to validate that faults (as per fault model) are not present. It has been shown in [9] that number of patterns required to perform structural testing is exponentially lower than that required for exhaustive functional testing. In Test Planning step, given a logic level circuit and fault model, we generate patterns, which when applied to a circuit determines that no fault from the fault model exists in the circuit.

Now we will illustrate test planning for the adder module of the example (Figure 8) assuming that fault model is "stuck-at". In "stuck-at" fault model each line of the circuit is assumed to have two types of faults i.e., s-a-0 and s-a-0. So if there are n lines in a circuit then in all there can be $2n$ stuck-at faults in the circuit. However, in the fault model it is assumed that only one stuck-at fault can occur at a time. In test planning we need to find input patterns which can determine that none of the stuck-at faults are present. In the circuit of Figure 8 as there are 12 lines (9 lines in circuit for "sum" and 3 lines in the circuit for "carry"), there can be 24 stuck-at faults. We take one fault at a time and determine an input pattern that can verify the absence of the fault. Here we will illustrate for only one fault and the same holds for all the other 23 faults. Let there be a stuck-at-0 fault in the output of one AND gate (shown in Figure 9) of the circuit for "sum". Now to we

need to find an input pattern that can verify the absence of the fault i.e., when the pattern in applied the output under normal and fault cases are different. Form Figure 9 it may be noted that if *a=1 and b=0* is applied as inputs, then "output1(sum)" is 0 if fault is present, 1 otherwise. So *a=1 and b=0* can verify the absence of fault by comparing output with 1.

It may be the fact that one pattern can test more than one fault and one fault can be tested by more than one pattern. Broadly speaking, determination of minimal test patterns which cover all faults is the most important step in test planning. Algorithms and techniques to perform test planning will be covered in "TESTING" part of the course.

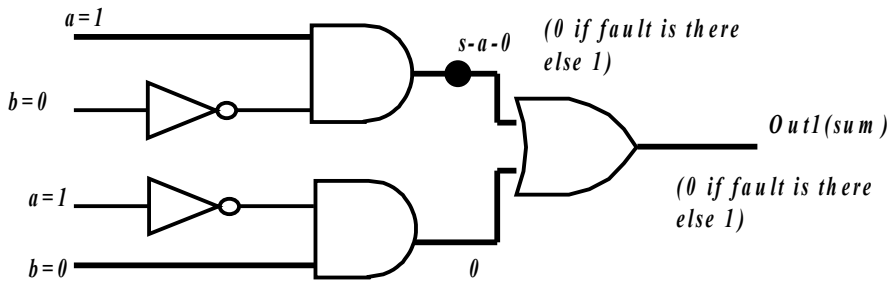


Figure 9. Detection of s-a-0 fault in the circuit for "*Out1(sum)"*

Step 6: Fabrication, Testing and Marketing.

Once all the steps from 1 through 5 are completed and verification after each level of transformations are done, the chips are fabricated, physically tested and fault free chips are sent for marketing.

The flow discussed above has been kept simple for introduction. In reality there are iterations back and forth, especially between any two adjacent steps, and occasionally even remotely separated steps. Although top-down design flow provides an excellent design process control, in reality, there is no truly unidirectional top-down design flow. Both top-down and bottom-up approaches have to be combined. For instance, if a chip designer obtained an RTL circuit from specifications (using HLS tools) without estimation of the corresponding chip area, then it is very likely that the resulting chip

layout exceeds the area limit. In such a case, in order to fit the architecture into the allowable chip area, the HLS design process must be repeated. Thus, it is very important to feed forward low-level information to higher levels (bottom up) as early as possible.

Further, details and integrities of the steps are also avoided. For example, there are some faults which cannot be tested by any pattern. For such cases, we need to put additional circuitry to make it testable called Design for Testability (DFT).

The respective modules in this course will elaborate on each of these aspects.

The breakup of the modules in this course is as follows:

Design

Module I: Introduction

Lecture I: Introduction to Digital VLSI Design Flow
Lecture II: High Level Design Representation
Lecture III: Transformations for High Level Synthesis

Module II: Scheduling, Allocation and Binding

Lecture I: Introduction to HLS: Scheduling, Allocation and Binding Problem
Lecture II and III: Scheduling Algorithms
Lecture IV: Binding and Allocation Algorithms

.

Module III: Logic Optimization and Synthesis

Lecture I,II and III: Two level Boolean Logic Synthesis
Lecture IV: Heuristic Minimization of Two-Level Circuits
Lecture V: Finite State Machine Synthesis
Lecture VI: Multilevel Implementation

Verification

Module - IV: Binary Decision Diagram

Lecture-I: Binary Decision Diagram: Introduction and construction
Lecture-II: Ordered Binary Decision Diagram
Lecture-III: Operations on Ordered Binary Decision Diagram
Lecture-IV: Ordered Binary Decision Diagram for Sequential Circuits

Module - V: Temporal Logic

Lecture-I: Introduction and Basic Operations on Temporal Logic
Lecture-II: Syntax and Semantics of CLT
Lecture-III: Equivalence between CTL Formulas

Module-VI: Model Checking

Lecture-I: Verification Techniques
Lecture-II, III and IV: Model Checking Algorithm
Lecture-V: Symbolic Model Checking

Test

Module VII: Introduction to Digital Testing

Lecture-I: Introduction to Digital VLSI Testing
Lecture-II: Functional and Structural Testing
Lecture-III: Fault Equivalence

Module VIII: Fault Simulation and Testability Measures

Lecture-I, II and III: Fault Simulation
Lecture-IV: Testability Measures (SCOAP)

Module IX: Combinational Circuit Test Pattern Generation

Lecture-I: Introduction to Automatic Test Pattern Generation (ATPG) and ATPG Algebras

Lecture-II and III: D-Algorithm

Module X: Sequential Circuit Testing and Scan Chains

Lecture-I: ATPG for Synchronous Sequential Circuits

Lecture-II and III: Scan Chain based Sequential Circuit Testing

○

Module XI: Built in Self test (BIST)

Lecture I and II: Built in Self Test

Lecture III and IV: Memory Testing

References

[1]. John P. Uyemura, , "CMOS Logic Circuit Design", Kluwer Academic Publishers, 1st Edition, 1999

[2]. Stephen M. Trimberger, "Introduction to CAD for VLSI", Kluwer Academic Publishers, 1st Edition, 1987

[3]., Daniel D. Gajski and Loganath Ramachandran, "Introduction to High-Level Synthesis", IEEE Design and. Test, volume 11,No, 4,1994, pp 44—54.

[4]. Daniel D. Gajski, Nikil D. Dutt, Allen C-H Wu, Steve Y-L Lin, "High-Level Synthesis: Introduction to Chip and System Design", Kluwer Academic Publishers, 1st Edition, 1992

[5]. Jr., Edmund M. Clarke, Orna Grumberg and Doron A Peled, "Model checking", MIT Press, 1st Edition, 1999.

[6]. Giovanni De Micheli, "Synthesis and Optimization of Digital Circuits", McGraw-Hill Higher Education, 1st Edition, 1994

[7]. Naveed A. Sherwani, "Algorithms for VLSI Physical Design Automation", Kluwer Academic Publishers, 2nd Edition, 1995

[8]. M. Abramovici, M.A. Breuer, and A.D. Friedman. Digital Systems Testing and Testable Design. Wiley-IEEE Press, 1994.